DAILY DATA DEEP DIVES · THE LATEST NEWS, FULLY EXPLAINED

Storage Wars

ClickHouse reads Postgres's raw diary, and Meta puts a doorman in front of ZippyDB

Issue #8 | Mon, 12 Oct 2026 | ⏱️ ~43 min read | Level 3/5

🗄️ **Today's two stories**

1. **ClickHouse open-sourced WalShadow, a Postgres-to-ClickHouse replicator that skips logical decoding and reads the same physical WAL a standby replica gets.** ClickHouse's own benchmark shows ~200 ms from commit to query, against ~10 s for its existing logical-replication tool. But a month of scrutiny (a Postgres consultant's critique, an independent teardown) has shown what you pay for that speed: you depend on byte formats Postgres never promised to keep stable, and you still need `wal_level = logical`. We explain the two kinds of WAL replication, how WalShadow decodes raw heap records with a "shadow" Postgres, and where the sharp edges are.
2. **Meta now routes more than a billion ZippyDB operations a second through ZGateway, a stateless proxy tier that cuts persistent connections ~19x.** It reverses a decade of "no extra hop" design at Meta. We explain why the direct client-to-shard mesh broke down (one routing bug put the fleet in a reboot loop), how the gateway batches, coalesces and sheds load per tenant, and what that means for your own PgBouncer, ProxySQL or Valkey proxy decisions.
3. **On the radar:** 8 more things that happened, listed at the end.

STORY 1 WalShadow: Postgres CDC without logical decoding

CLICKHOUSE WALSHADOW · POSTGRES → CLICKHOUSE · ANNOUNCED 10 SEP 2026 · CRITIQUES 22 SEP AND AFTER · ~21 MIN READ

Every Postgres server keeps a diary. Before it changes a single byte on disk, it writes down what it's about to do, and a standby replica in another data centre stays in sync by reading that diary, page by page. For a decade, anyone who wanted those changes in *another* kind of system, a warehouse, a search index, a Kafka topic, had to ask Postgres to translate the diary first, through a mechanism called logical decoding. ClickHouse has now shipped a tool that reads the diary in the original, raw bytes and all, and translates it somewhere else. It's fast. It's also a bet on handwriting that was never meant for outside readers.

WHAT HAPPENED

On **10 September 2026**, ClickHouse published "Introducing WalShadow: Sub-second Postgres replication to ClickHouse from physical WAL", written by Sai Srirampur. WalShadow is described as "an open-source engine that replicates Postgres data to ClickHouse directly from the physical WAL". It doesn't use Postgres's logical replication. Instead it consumes the same physical WAL stream that Postgres replicas use, decodes it outside the source database, and writes ClickHouse-native blocks straight into ClickHouse, "effectively turning ClickHouse into an analytical physical standby".[1]

The code is on GitHub at `ClickHouse/walshadow` under the **AGPL-3.0** licence. The repository has no tagged releases yet (173 commits on `main` when we looked).[3] A managed version is in **private preview** on ClickHouse Managed Postgres, and ClickHouse's product page still labels it "Private Preview".[2] The blog post is clear that this is early: over "the coming months" ClickHouse will harden it with an initial group of design partners.[1]

The headline numbers are ClickHouse's own, from a benchmark streaming changes into a single table, with everything in one region on `c8i.2xlarge` (8 vCPU) instances:[1]

- **Commit-to-visible latency:** about **200 ms** for WalShadow, against about **50 ms** for a native Postgres physical replica (the floor) and about **10 seconds** for PeerDB, ClickHouse's own logical-replication tool that powers its ClickPipes service.
- **Throughput:** the source Postgres maxed out at about **290,000** inserted rows per second; WalShadow kept up at **289,000**; PeerDB managed about **120,000**.

The story didn't stop at the launch post. On 11 September the post reached Hacker News, where one commenter praised the choice of `wal_level = replica` instead of `logical`.[4] That turned out to be a misreading, and a useful one, as we'll see. On **22 September**, Christophe Pettus, a long-time Postgres consultant (PGX Inc.), published "The Shadow Knows", accepting the numbers but arguing that WalShadow "is escaping logical decoding's *decoder* while still consuming logical decoding's *WAL*", and listing the problems it now has to solve alone.[5] And an independent teardown on GitHub, `walshadow-101`, built a toy WAL decoder to test the claims, calling the idea "real" and the "50x faster than PeerDB" headline "a vendor benchmark against their own product".[6]

So this is the right moment to explain it properly: the speed is real enough to matter, the open questions are now well documented, and Postgres 19, due this month, changes one of the knobs involved.

THE BACKGROUND YOU NEED

The diary: write-ahead logging

Postgres stores each table as a **heap** ≈ *Postgres's name for a table's main data file: an unordered pile of 8 kB pages, each holding row versions (tuples)*, a set of 8 kB pages that hold **tuples** ≈ *Postgres's physical row versions; an UPDATE writes a new tuple and marks the old one dead rather than editing in place*. Changing a page directly would be dangerous: if the server crashed halfway through writing it, you'd be left with a torn page. So Postgres writes every change to the **WAL** ≈ *write-ahead log: an append-only journal where Postgres records every change before applying it to data files, so it can replay the log after a crash* first. Every record in it has a position, the **LSN** ≈ *log sequence number: a record's byte position in the WAL, which only ever grows, so it doubles as a timestamp for "how far along" a server is*.

The WAL is physical. A record says, roughly, "in relation file 16384, block 7, insert these tuple bytes at offset 3". It doesn't say "table `users`" or "column `email`". It's written for one reader: Postgres itself, during crash recovery.

There's a subtlety that matters later. The first time a page is modified after a **checkpoint** *≈ a moment when Postgres flushes all dirty pages to disk, so crash recovery only has to replay WAL written after it*, Postgres may log the *entire page*, a **full-page image** *≈ a copy of a whole 8 kB page written into the WAL on its first change after a checkpoint, so recovery can rebuild a page that was torn mid-write*. The docs explain why: a page write in progress during a crash "might be only partially completed", and "the row-level change data normally stored in WAL will not be enough to completely restore such a page". The `full_page_writes` setting is on by default.[7] For a third-party reader, a full-page image is awkward: the change you care about is buried inside 8 kB of page bytes.

Physical replication: ship the diary

Postgres's built-in **streaming replication** *≈ sending WAL records from a primary to standby servers as they're written, so standbys replay them and stay byte-for-byte identical* simply ships these WAL records to a **standby** *≈ a replica server that continuously replays the primary's WAL; it can serve read-only queries but not writes*. The standby replays them exactly as crash recovery would, so it ends up byte-identical to the primary. This is cheap for the primary (it already writes the WAL; it just sends it over the network) and very fast. It's the 50 ms floor in ClickHouse's chart.

But a physical standby has to be Postgres, of the same major version, on a compatible platform. You can't point a physical WAL stream at Kafka or ClickHouse and expect it to understand.

Logical decoding: translate the diary

So in Postgres 9.4 (2014), the project added a translation layer.[9] The docs define it: "Logical decoding is the process of extracting all persistent changes to a database's tables into a coherent, easy to understand format which can be interpreted without detailed knowledge of the database's internal state." It works "by decoding the contents of the write-ahead log, which describe changes on a storage level, into an application-specific form such as a stream of tuples or SQL statements."[8]

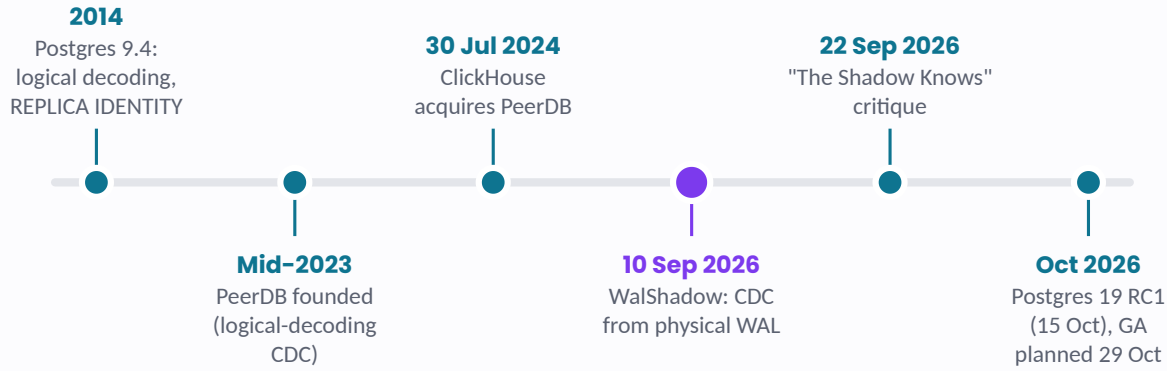
Three pieces make it work:

- **Output plugins** *≈ code loaded into Postgres that turns decoded changes into the format a consumer wants (pgoutput for built-in logical replication, wal2json, etc.)* "transform the data from the write-ahead log's internal representation into the format the consumer of a replication slot desires".[8]
- A **replication slot** *≈ a named, crash-safe bookmark on the primary that remembers how far a consumer has read and stops Postgres from deleting WAL (and, for logical slots, old catalog rows) it still needs* represents "a stream of changes that can be replayed to a client in the order they were made". Slots "will prevent removal of required resources even when there is no connection using them", meaning neither WAL "nor required rows from the system catalogs can be removed by `VACUUM`" while a slot needs them.[8]
- An **exported snapshot** *≈ a consistent point-in-time view of the database that a new slot hands out, so a tool can copy existing rows and know that every later change will arrive through the slot* lets a tool copy the existing data and then stream changes with no gap and no overlap.[8]

Logical decoding also needs extra information in the WAL, which is what `wal_level` controls. The default, `replica`, "writes enough data to support WAL archiving and replication". `logical` "adds information necessary to support logical decoding".[7] Part of that extra information is the old row's key for updates and deletes, controlled per table by **REPLICA IDENTITY** *≈ a per-table setting deciding which old-row columns Postgres writes to the WAL for UPDATE/DELETE: the primary key (DEFAULT), a chosen unique index, all columns (FULL) or nothing*.[9,10] Without it, a consumer seeing "row deleted" has no idea *which* row.

Postgres **CDC** *≈ change data capture: streaming every insert, update and delete from a database's log so another system can mirror it* tools are typically built as logical-decoding clients, and PeerDB is one of them. PeerDB was founded in mid-2023 by Kaushik Iska and Sai Srirampur, and ClickHouse bought it on 30 July 2024 to become its native Postgres CDC.[11] That's why Srirampur, who wrote the WalShadow post, calls the benchmark against PeerDB "bittersweet": he's beating his own previous product.[1]

Getting changes out of Postgres



Sources: Paquier on 9.4, ClickHouse on PeerDB, WalShadow post, The Build, PG 19 open items.

Why logical decoding can be slow, and where it hurts

Logical decoding runs *inside* the source database. A walsender process reads WAL, reassembles each transaction (WAL interleaves records from many concurrent transactions), hands committed changes to the output plugin, and ships the result. The consumer then typically deserialises it, often into JSON or Avro, maybe via Kafka, before loading it into the target. Each hop costs time, and the decoding work costs CPU on your production primary.

Logical slots also have an operational sting that every Postgres on-call engineer learns once. Because a logical slot holds back both WAL and old catalog rows until the consumer confirms it has read them, a stalled CDC consumer makes the primary's disk fill up with WAL and blocks some `VACUUM` clean-up. "In extreme cases this could cause the database to shut down to prevent transaction ID wraparound," the docs warn.[8]

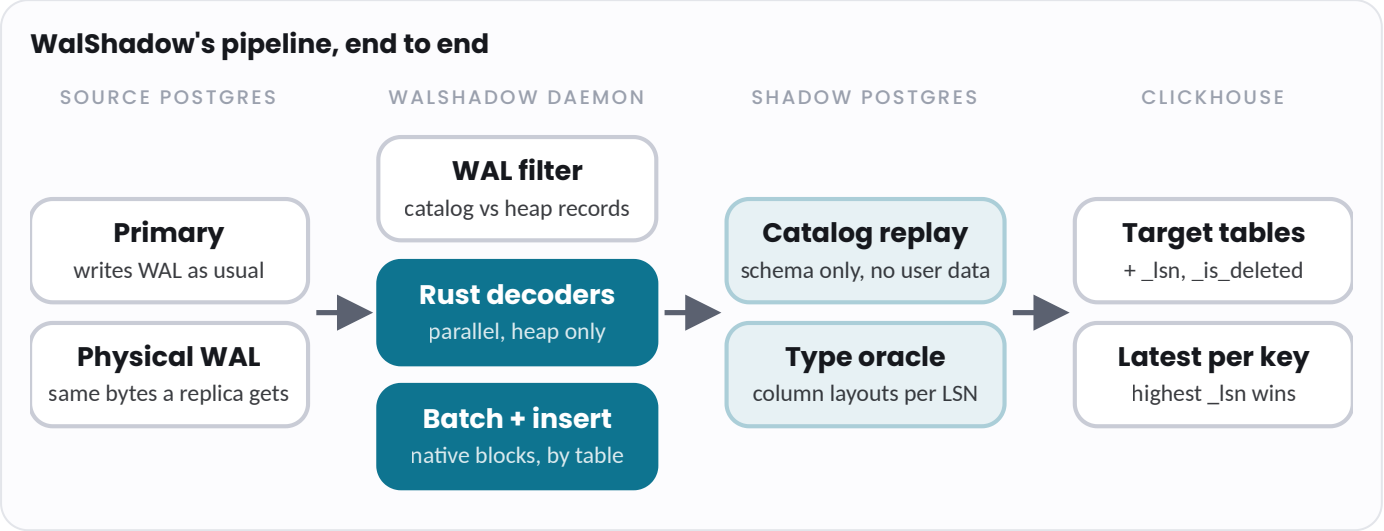
WalShadow's pitch is aimed precisely at these two pains: no decoding on the primary, and no logical slot.

HOW IT WORKS

Four stages, one shadow

The launch post describes four stages:[1]

1. **Track the live schema.** WalShadow filters the WAL records that touch the system **catalog** *≈ Postgres's internal tables (pg_class, pg_attribute, pg_type…) describing every table, column and type* and replays them into a "schema-only shadow Postgres instance". That shadow always knows what tables and columns looked like at the current point in the WAL.
2. **Decode data changes in parallel.** Heap records go to a pool of Rust decoders. They never pass through the shadow.
3. **Build ClickHouse-native blocks.** A batcher groups decoded rows by table into ClickHouse's own block format, with no intermediate JSON or Avro.
4. **Insert in parallel.** A separate pool of inserters writes several blocks concurrently, so decoding and insertion scale independently.



The shadow is the clever part, and both critics agree on that. The teardown calls it "Real, and the best idea here".[6] Here's why it's needed. A raw heap record is a run of bytes. To turn it into `(id=42, email='a@b.c', plan='pro')` you must know, *as of that exact LSN*, how many columns the table had, their types, their alignment, which were dropped. The teardown's demo makes the point vividly: it decodes an `INSERT` correctly with the right catalog, then decodes the same bytes with a stale catalog after an `int → bigint` column change, and gets garbage. The README calls that "the whole reason for the shadow".[6]

Postgres's own logical decoding solves this with a "historic" catalog snapshot inside the server. WalShadow solves it by running a second Postgres, of the same major version as the source, bootstrapped from a **base backup** *≈ a full physical copy of a Postgres data directory, the starting point for building a standby* and fed only the catalog part of the WAL. Pettus describes how schema changes are handled: at a schema boundary, the pipeline pauses publication until the shadow reaches that exact replay position, then reads and saves the **tuple descriptors** *≈ Postgres's in-memory description of a row's layout: column count, types, lengths and alignment* that the parallel decoders use.[5] The launch post adds that schema changes and truncations "act as barriers": they wait until all earlier data is durable in ClickHouse.[1]

There's one more reason the shadow must be a real Postgres. "Half the type system is defined by C functions in the server," Pettus writes: `jsonb`, arrays, enums, ranges, domains and extension types are converted by the shadow server, so you must install the source's extensions on the WalShadow host too.[5] The README backs this up: a native module in `pgext/` has to be built and loaded into the shadow through `shared_preload_libraries`.[3]

Out-of-order blocks, made consistent by the LSN

Parallel inserters mean blocks can land in ClickHouse out of order. WalShadow handles this by stamping every row with its source WAL position in an `_lsn` column, and ClickHouse "uses `_lsn` to retain the latest version for each key".[1] Pettus lists the full set of extra columns: `_lsn`, `_xid`, `_commit_ts` and `_is_deleted`, with tables keyed on the source's replica identity.[5]

If you've used ClickHouse, that shape is familiar. The **ReplacingMergeTree** *≈ a ClickHouse table engine that keeps only the newest row per sorting key, using an optional version column, and drops older ones during background merges* engine keeps, for rows with the same sorting key, the one with the highest `ver` column, and supports an `is_deleted` flag. Deduplication happens only during background merges "at an unknown time", so a query that must be exact uses **FINAL** *≈ a ClickHouse query modifier that applies deduplication at read time, so results are correct even before background merges finish*.[12] (The launch post doesn't name the engine WalShadow creates; the match to ReplacingMergeTree's contract is our reading.) The practical upshot: "200 ms to visible" means the new row version is in ClickHouse within 200 ms. Whether your query sees *only* the new version depends on whether you query with `FINAL` or the merge has happened.

The catch hidden in `wal_level`

Now back to that Hacker News comment. A reader admired WalShadow for using `wal_level = replica` rather than the `logical` setting CDC tools normally need.[4] The README says otherwise. Its preflight checks, enforced at start-up, require:[3]

- Postgres **16 or later** (`server_version_num >= 160_000`), and a shadow of the **same major version**.
- `wal_level = logical`.
- A row key for deletes on every replicated table: a primary key (`REPLICA IDENTITY DEFAULT`), `USING INDEX` or `FULL`. `NOTHING` and key-less `DEFAULT` are rejected.
- Optionally, `--slot`, which must name an existing **physical** replication slot.

Why `logical`, if WalShadow never decodes logically? Because the extra information that `wal_level = logical` adds is exactly what a third-party decoder needs. The teardown demonstrates it: with `wal_level=replica`, "DELETE loses its key and UPDATE loses its tuple".[6] At the `replica` level, Postgres writes only what *Postgres* needs to redo a change on an identical copy of the page, which can mean a delete record that identifies a slot on a page rather than a key, or new tuple data that lives inside a full-page image. At `logical`, the old key (per REPLICA IDENTITY) and complete new tuple data are logged. (That's our explanation of the mechanism; the teardown demonstrates the effect.) So the teardown rates the "no logical replication

overhead" claim "Half true": there are no logical slots, but you still pay for logical-level WAL.[6] Or in Pettus's words, it still consumes logical decoding's WAL.[5]

Typical logical CDC setup vs WalShadow's preflight

Logical-decoding CDC (typical)

```
-- needs a restart
ALTER SYSTEM SET wal_level = logical;
-- each table needs a key
ALTER TABLE users
  REPLICA IDENTITY DEFAULT;
CREATE PUBLICATION cdc_pub
  FOR TABLE users;
SELECT pg_create_logical_replication_slot(
  'cdc', 'pgoutput');
```

WalShadow (per README checks)

```
-- still required
ALTER SYSTEM SET wal_level = logical;
-- still required: a row key
ALTER TABLE users
  REPLICA IDENTITY DEFAULT;
-- no publication, no logical slot;
-- optional physical slot (--slot)
SELECT
  pg_create_physical_replication_slot(
  'walshadow');
```

Left is a generic pgoutput-based setup for illustration; individual CDC tools differ. Right reflects the README's preflight requirements; slot name is illustrative.

Postgres 19 adds a twist. Its docs say that with `wal_level = replica`, "The system automatically increases the effective WAL level to `logical` when creating the first logical replication slot, and decreases it back to `replica` when dropping or invalidating the last logical replication slot".[7] That's great for ordinary CDC tools: no restart needed to turn logical decoding on. But WalShadow has no logical slot, so (our inference) on Postgres 19 you'd still set `wal_level = logical` by hand, and WalShadow would first have to be audited for the new major version at all.

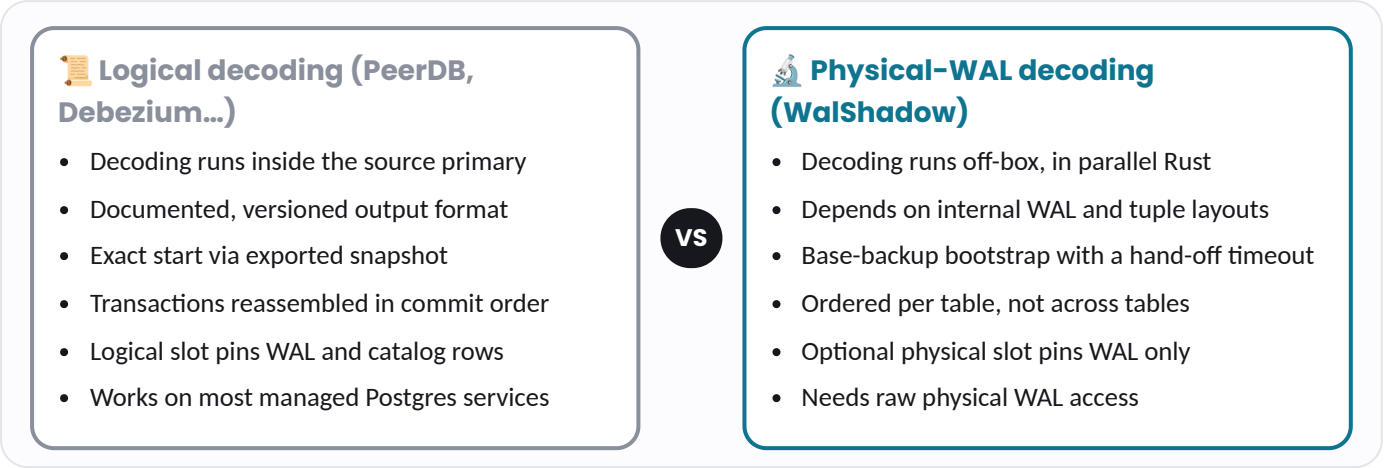
What WalShadow had to rebuild

Pettus's central argument is that logical decoding isn't just a slow translator. It's a set of hard-won correctness machinery, and a physical-WAL reader has to rebuild each piece outside the server:[5]

- **Transaction reassembly.** WAL interleaves concurrent transactions and **subtransactions** *≈ nested transactions created by SAVEPOINTs or exception blocks, which can roll back independently of their parent*. Postgres's `reorderbuffer.c` reassembles them and spills big ones to disk. WalShadow has its own buffer and spill directory (the `--spill-dir` flag in the README).[3] But its output is ordered per table, so "updates from different tables inside one PostgreSQL transaction may become visible in ClickHouse at different moments".
- **Catalog visibility.** **Command IDs** *≈ counters (cmin/cmax) that order statements within one transaction so it sees its own earlier changes* aren't written to WAL in their normal form; logical decoding gets them from special `XLOG_HEAP2_NEW_CID` records. The shadow side-steps this.
- **TOAST.** Large values are stored out of line by **TOAST** *≈ The Oversized-Attribute Storage Technique: Postgres compresses large values or splits them into ~2 kB chunks in a side table, keeping only a pointer in the row*. A row wider than about 2 kB gets values moved out into a side table keyed by `chunk_id` and `chunk_seq`, and an UPDATE

"incurs no TOAST costs if none of the out-of-line values change", so those chunks aren't rewritten.[13] That's the hard case: logical replication can send an "unchanged" marker because the subscriber already has the value. WalShadow must resolve the pointer itself, so it keeps an optional "TOAST mirror" of chunks. Pettus warns that reused TOAST value IDs can leave "ambiguous generations", and the failure mode is quiet: "a plausible older value with nothing to tell you".

- **Initial load.** Logical slots give an exact start point through the exported snapshot. WalShadow bootstraps from a base backup and can miss rows or keep deleted ones if transactions are still open at hand-off; the mitigation is a timeout. Pettus quotes WalShadow's own docs: "Increasing handoff wait reduces exposure without proving completeness."
- **WAL retention.** A physical slot still pins WAL just like a logical one, so you still size `wal_keep_size` or keep archives. What you do avoid is the **catalog_xmin** *≈ the oldest transaction whose catalog rows a logical slot still needs; it stops VACUUM from cleaning those catalog rows* hold that logical slots impose.



The stability bet

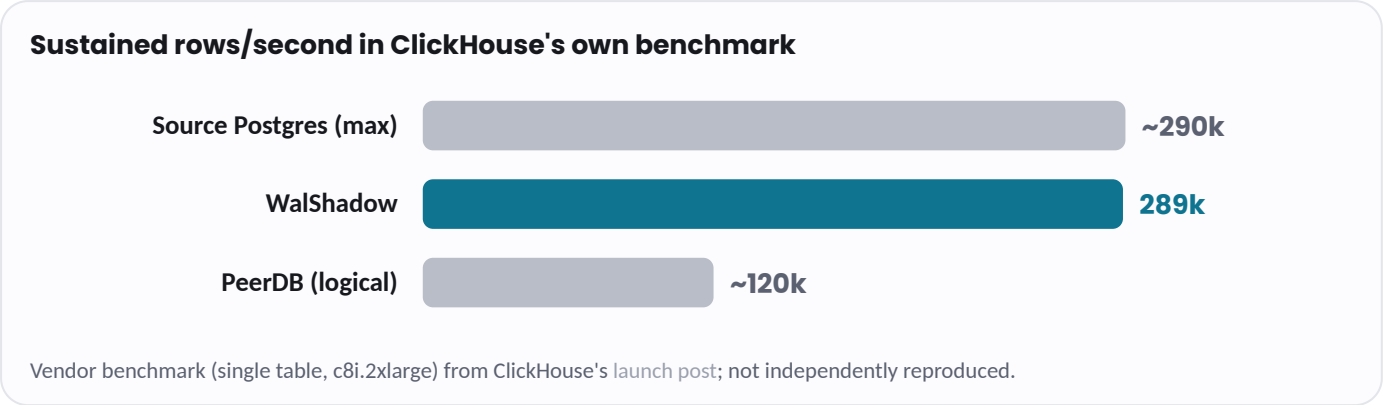
The deepest objection is about contracts. Logical replication's wire format is a documented, versioned interface. The physical WAL layout and heap tuple format are implementation details that change between major versions. Pettus points to a 2026 rename of an internal TOAST structure, `varatt_external` to `varatt_external_oid`, as an example of those internals moving in public, and notes that extensions can register custom **resource managers** *≈ WAL "record types" owners; extensions can add their own, writing records an outside decoder has never seen* that write records an outside decoder has never seen. WalShadow therefore refuses to run against major versions it hasn't audited, which means your Postgres upgrade schedule depends on ClickHouse's audit schedule.[5]

He also quotes the project's own limitation docs, which he praises as "better written than most of the code", including the line "Unsupported behavior is not uniformly rejected at startup". Prepared transactions, non-default tablespaces during bootstrap and unplanned primary promotion aren't production-ready.[5] The teardown, assessing version **v0.1.2**, sums up: "a strong architecture with an early implementation".[6]

WHAT IT MEANS FOR YOU

If you're on RDS, Aurora, Cloud SQL or Supabase, you can't use it today. Most managed Postgres services don't expose physical WAL to customers, as ClickHouse's own post acknowledges, which is why the managed version runs on ClickHouse Managed Postgres, where ClickHouse controls the replication layer.[1] The teardown names RDS, Aurora, Cloud SQL and Supabase explicitly.[6] (One Hacker News commenter asked about Supabase for exactly this reason.[4])

If you self-host Postgres 16 or newer and feed ClickHouse, WalShadow is worth a staging trial when the gap between "it happened in Postgres" and "the dashboard shows it" matters: fraud signals, live operational dashboards, user-facing analytics. Treat the numbers as a ceiling, not a promise. ClickHouse says performance "will vary by workload", and its benchmark used one table.[1] The teardown measured about 0.8–1.1 seconds on a laptop across five tries, versus the claimed ~200 ms, calling that a "Sanity check, not a benchmark".[6] Both are far better than 10 seconds.



Check the licence. WalShadow is AGPL-3.0.[3] If you'd embed it in something you distribute or offer as a service, ask your legal team first.

Design for per-table consistency. If your analytics join `orders` to `order_items` and both changed in one Postgres transaction, WalShadow may show the order before its items for a moment.[5]

Plan upgrades around it. A tool that refuses unaudited major versions turns "upgrade Postgres" into "upgrade Postgres after WalShadow supports it". With Postgres 19 RC1 scheduled for 15 October and GA planned for 29 October, that's a real question for early adopters.[14]

💥 Watch out for

- `wal_level = logical` **is still required:** no logical *slots*, but not less WAL.[3,6]
- **Vendor benchmark, one table:** ClickHouse versus its own PeerDB.[1]
- **Silent divergence:** TOAST mirror and bootstrap gaps give plausible wrong data, not errors.[5]
- **Newest version needs** `FINAL` **or merges:** older row versions can linger until merged.[12]
- **Early software:** no tagged GitHub release; managed version in private preview.[3]

WHAT HAPPENS NEXT

The obvious next milestones are a first tagged release, Postgres 19 support (19 is due to reach GA around 29 October), and the managed version leaving private preview. The broader question is whether the Postgres project ever offers a middle ground: a stable, documented "row image" stream that's cheaper than full logical decoding. Postgres 19's automatic effective WAL level shows the project is willing to make logical replication easier to switch on.[7] Until then, WalShadow is an impressive demonstration of how much latency logical decoding's machinery costs, and an equally clear list of what that machinery was protecting you from.

📖 Words from this story

Heap: Postgres's name for a table's main data file: an unordered pile of 8 kB pages holding row versions.

Tuples: Postgres's physical row versions; an UPDATE writes a new one and marks the old one dead.

WAL: write-ahead log, the append-only journal where Postgres records every change before applying it.

LSN: log sequence number, a record's ever-growing byte position in the WAL.

Checkpoint: a moment when Postgres flushes dirty pages, so recovery only replays WAL written after it.

Full-page image: a whole 8 kB page copied into the WAL on its first change after a checkpoint.

Streaming replication: shipping WAL from a primary to standbys as it's written.

Standby: a replica that continuously replays the primary's WAL and can serve reads.

Output plugins: code inside Postgres that formats decoded changes for a consumer.

Replication slot: a crash-safe bookmark that stops Postgres deleting WAL (and, for logical slots, catalog rows) a consumer still needs.

Exported snapshot: a consistent point-in-time view handed out by a new slot, for a gap-free initial copy.

REPLICA IDENTITY: per-table setting for which old-row columns are written to WAL on UPDATE/DELETE.

CDC: change data capture, streaming every insert, update and delete to another system.

Catalog: Postgres's internal tables describing tables, columns and types.

Base backup: a full physical copy of a data directory, used to seed a standby.

Tuple descriptors: the description of a row's layout (columns, types, alignment).

ReplacingMergeTree: ClickHouse engine that keeps the newest row per key, deduplicating during merges.

FINAL: ClickHouse modifier that deduplicates at query time.

Subtransactions: nested transactions from SAVEPOINTs or exception blocks.

Command IDs: counters ordering statements within a transaction.

TOAST: Postgres's storage for large values: compressed or split into chunks in a side table.

catalog_xmin: the oldest transaction whose catalog rows a logical slot still needs.

Resource managers: owners of WAL record types; extensions can add their own.

STORY 2

Meta puts a gateway in front of ZippyDB

ARCHITECTURE CASE STUDY · META ZIPPYDB → ZGATEWAY · META BLOG 3 SEP 2026, INFOQ 28 SEP · ~17 MIN READ

For more than a decade, Meta's infrastructure has favoured one rule: don't put a proxy in the middle. Services talk to each other through a library linked straight into the binary, so there's no extra hop and no extra fleet to run. Then, one day, a routing bug made every ZippyDB client open a connection per shard, the database hosts ran out of file descriptors, and the fleet fell into a reboot loop. Today more than a billion operations a second to Meta's most widely used key-value store go through exactly the kind of middle tier the rule was meant to avoid.

WHAT HAPPENED

On **3 September 2026**, Meta engineers Rittik Banik and Yunhao Cao published "ZGateway: Learnings from Putting a Proxy in Front of ZippyDB", a description of a stateless proxy tier that now sits between **ZippyDB** ≈ *Meta's general-purpose, strongly consistent distributed key-value store, built on RocksDB and in production since 2013* clients and the database servers (called ZServers). Meta calls ZippyDB "the most widely used key value store at Meta", backing product metadata, counters and configuration.[15] InfoQ covered it on **28 September**.[16]

The key facts, all from Meta's post:[15]

- ZGateway handles **more than 1 billion operations per second**, about **40%** of ZippyDB traffic, projected to exceed **60%**. InfoQ adds that the stated goal is to route all ZippyDB traffic through it.[16]
- Meta's model of the fleet estimates roughly **19x fewer** persistent connections overall and a **97–98%** drop in per-host connection counts. InfoQ stresses these are model estimates, not measurements.[16]
- The gateway costs roughly **6%** in computational overhead for an average use case.
- In a controlled overload test, per-tenant load shedding confined the damage to **6 of about 1,350** tenants, while the rest ran 99.9% of requests with zero rejections.

That last number hints that this is more than a connection pooler. To see why Meta built it, and why it took so long, we need to start with how ZippyDB clients talked to the database before.

THE BACKGROUND YOU NEED

ZippyDB in one page

ZippyDB was first deployed in 2013, to stop every team from running its own **RocksDB** ≈ *an embedded key-value storage engine from Meta, based on LSM trees, that stores data on local SSD* and re-implementing replication and failover each time.[17] Data is split into **shards** ≈ *independent slices of the keyspace, each stored and replicated separately; ZippyDB's are typically 50–100 GB*, typically 50–100 GB each, and each shard is replicated across regions.[17]

For each shard, a subset of replicas forms a **Paxos** ≈ *a consensus protocol that lets a group of replicas agree on the order of writes even if some fail; a write is durable once a majority accepts it* quorum, and writes are replicated synchronously through Multi-Paxos. Other replicas are followers that receive data asynchronously and serve low-latency reads "with relaxed consistency". Time is divided into epochs, each with a single leader that orders all writes. Clients choose their consistency per read: eventual (which Meta says is "closer to bounded staleness"), read-

your-writes (the client caches the latest sequence number it wrote), or strong reads from the primary.[17] A busy service can therefore need to reach many different replicas of many different shards.

The direct-access tradition

Meta's service-to-service traffic runs on **ServiceRouter** ≈ *Meta's global service mesh, in production since 2012, implemented mainly as a routing library linked into each service rather than as sidecar proxies*. Its OSDI 2023 paper describes "millions of L7 routers" routing "tens of billions of requests per second", using "a routing library that is directly linked into service executables". Sidecar or remote proxies carry about 1% of RPC requests; the library carries the rest.[18] The paper's title sums up the goal, a "Minimal Cost Service Mesh": the linked library lets clients reach servers without an extra proxy hop.

ZippyDB followed the same model, with a **thick client** ≈ *a client library that contains real database logic (routing, shard maps, replica selection, batching, retries) instead of just sending requests to a server*. Each client process figured out which shard a key lived on, which replica to call, and opened its own connections to those database hosts. The post describes the result as a mesh: "each client connects to every database host it needs".[15]

Why a mesh of connections hurts

A network connection isn't free on either end. Each one holds a **file descriptor** ≈ *an operating-system handle for an open file or socket; each process has a hard limit on how many it may hold*, memory for buffers and TLS state, and some CPU for keep-alives. Meta's numbers make the scale clear: a client can touch tens of thousands of shards in a stable window, spread across hundreds of thousands of hosts, so "a typical client holds tens of thousands of outbound connections, and a typical database host accepts tens of thousands of inbound ones".[15]

Worse, inbound connections on a database host grow with the *client* population, which the database team doesn't control. A dip in connection reuse can trigger a connection storm. Meta ties host crashes from file-descriptor exhaustion and **OOMs** ≈ *out-of-memory kills: the operating system terminating a process that has exhausted available memory* to this pattern, and describes the worst case: a routing bug made every client open a connection per shard, hosts breached their file-descriptor limits, and the fleet entered a reboot loop.[15]

INCIDENT PATTERN (as described by Meta)

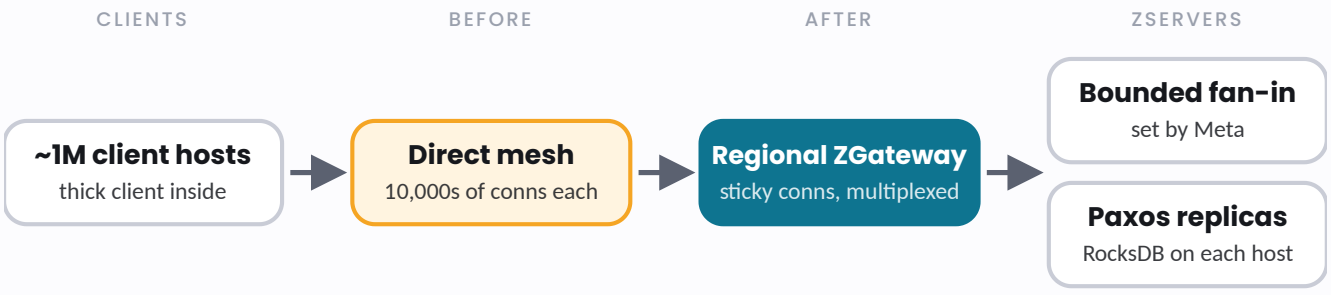
```
trigger      routing bug in client fleet → one connection per shard
effect       inbound connections per DB host spike
limit hit    file-descriptor limits breached on hosts
result       fleet enters a reboot loop
root cause   fan-in scales with client count, not with anything the DB team controls
```

And fixing it in the client is slow. Client fleets and the database fleet deploy on different schedules, so a fix in the client library only works once every client has picked it up. Meta's conclusion is that a separate tier was needed to decouple the two.[15]

If this sounds familiar, it should. It's the same problem that makes Postgres teams put **PgBouncer** ≈ *a lightweight connection pooler for Postgres that lets many client connections share a small number of server connections* in front of the database, and it's a cousin of what Percona's Valkey-proxy, now being contributed to the Valkey project,

does for apps written against a single Redis instance that need to talk to a Valkey Cluster.[19] InfoQ makes the general point: connection poolers, service meshes, CDNs and API gateways all use the same pattern with different trade-offs.[16]

From a client mesh to a gateway tier



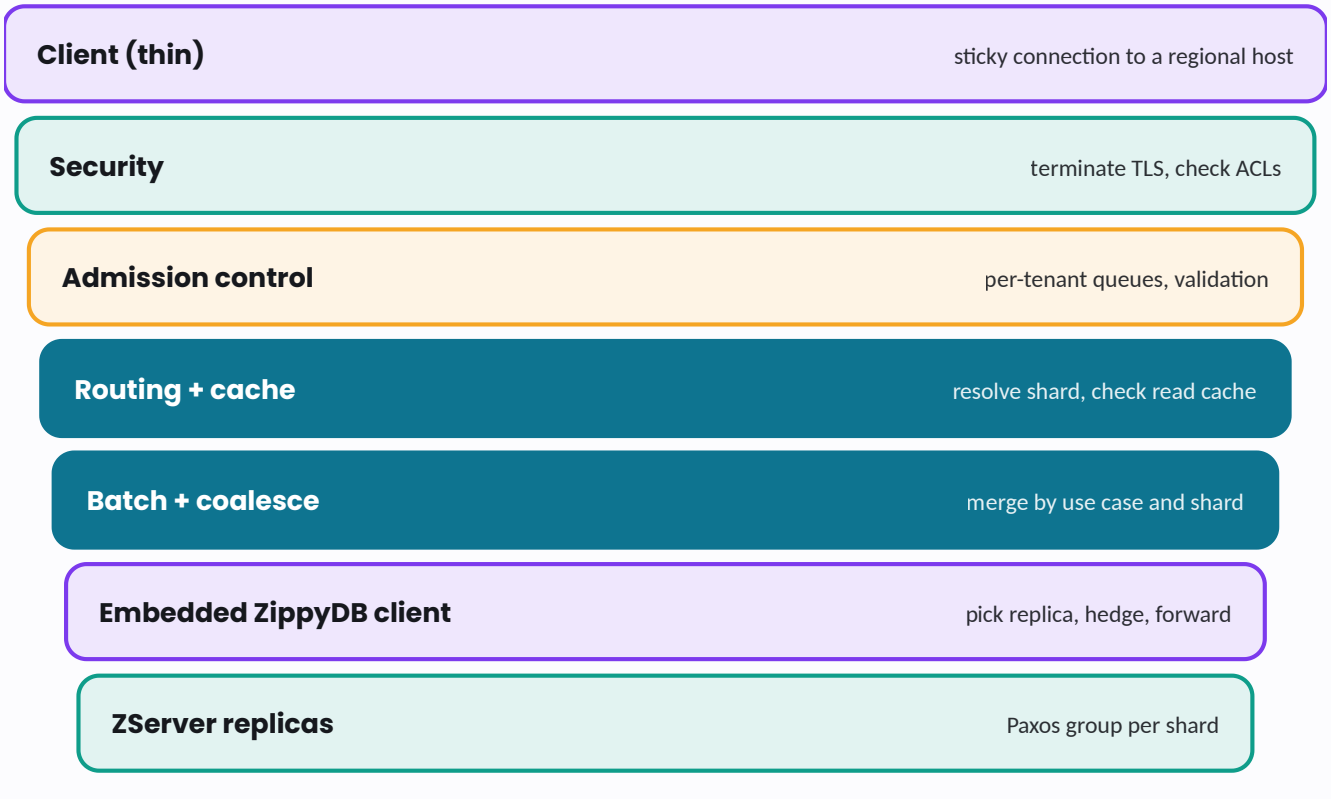
Client count is from Meta's mock model (1,000,000 clients); connection descriptions from Meta's post.

HOW IT WORKS

The request path

ZGateway is a stateless proxy tier deployed in regional tiers, which clients discover through ServiceRouter so they stay near their gateway. Inside, it runs Meta's existing C++ thick client, one internal client per use case, so all the hard routing logic lives on in one place instead of being reimplemented.[15] The post walks through what happens to a request:[15]

One request through ZGateway



Two terms in that stack deserve a definition. **Admission control** ≈ *deciding at the front door whether to accept a request at all, based on current load and the caller's quota, instead of letting overload reach the database* is the gateway's first line of defence. **Hedging** ≈ *sending a duplicate request to a second replica if the first is slow, and using whichever answers first, to cut tail latency* is a classic tail-latency trick that, per the post, stays in the existing Thrift/ServiceRouter stack and embedded client, along with TLS, key-to-shard mapping and replica selection. ZGateway owns traffic management.[15]

The connection maths

Why does adding a tier *reduce* connections? Because each backend connection from a gateway host now carries requests from many clients. Meta models it as "balls into bins" with mock figures: 20 regions, 500,000 database hosts, 30,000 proxy hosts, 1,000,000 clients and 50,000 shards per client. The model gives the ~97–98% per-host and ~19x total reductions.[15]

The more important change, in Meta's words, is the *scaling behaviour*. Fan-in on a database host no longer grows with the number of clients. It depends on the number of regions and the shard density per host, numbers the ZippyDB team sets itself: "An unbounded number driven by everyone else becomes a bounded number we control."[15]

Batching and coalescing: seeing everyone's traffic

A thick client can only batch requests from its own process. A gateway host sees requests from thousands of clients, which opens two optimisations:[15]

- **Batching.** A shared batcher on each host groups requests by use case and physical shard and merges them into one backend RPC. A batch flushes when a **linger window** ≈ *a short wait during which a batcher collects more requests before sending, trading a little latency for fewer, larger calls* elapses, a size limit is reached or a request-count cap is hit. Oversized or just-migrated batches fall back to individual sends.
- **Coalescing** ≈ ***merging identical concurrent requests (same key) into one backend fetch and sharing the answer with every caller*****.** When many callers ask for the same key at once, only one fetch goes to the database. That protects against **hot-key** ≈ *a single key receiving a disproportionate share of traffic, overloading the shard that owns it* stampedes.

Two safeguards keep the batcher from becoming a memory problem: idle eviction of stale batch entries, and an in-flight cap that rejects new executions during overload. A side benefit: Meta could retire "fragile, CPU-heavy client-side batching libraries".[15] If you've tuned Kafka's `linger.ms`, it's the same trade: wait a little, send fewer, bigger requests.

🧳 Thick client, direct mesh

- No extra network hop
- Batching only within one process
- DB fan-in grows with client count
- Fixes ship at every client's deploy pace
- Per-client connection and TLS cost

VS

🚪 ZGateway tier

- One extra hop, ~6% compute overhead
- Batches and coalesces across clients
- Fan-in bounded by regions and shards
- One fleet to fix, observe and roll back
- Per-tenant admission and load shedding

Noisy neighbours: Discriminant Load Shedding

A shared tier has a classic risk: one misbehaving **tenant** ≈ *one customer of a shared system; at Meta, a use case or service sharing ZippyDB capacity with others* can starve everyone else. ZGateway's answer, which Meta calls Discriminant Load Shedding, gives each tenant bucket its own queue and drains the queues round-robin. When the host is overloaded, the tenants sending too much get shed first.[15]

Meta's test numbers: above 90% CPU with about 1,350 active buckets, only 6 noisy tenants were shedding. The remaining ~1,344 executed 99.9% of their requests with zero rejections, and **goodput** ≈ *the rate of useful, successful work, as opposed to raw throughput that includes rejected or failed requests* held near 97–98%. The machinery itself cost about 8% of CPU.[15]

ZGateway in Meta's numbers

1B+	~40%	~19x	97–98%	~6%	6 of ~1,350
operations per second through the gateway	of ZippyDB traffic (projected **>60%**)	fewer persistent connections (model)	fewer connections per host (model)	compute overhead, average use case	tenants shed in an overload test

All figures as reported by Meta (engineering blog, 3 Sep 2026); the connection figures come from a model with mock inputs, not measurements.

Caching, balancing and moving transactions

ZGateway comes in two flavours that share one pipeline: a pure proxy and a **read-through cache** ≈ *a cache that, on a miss, fetches the value from the database itself and stores it, so callers only ever talk to the cache*. The cache uses per-key fill locks to prevent **thundering herds** ≈ *many requests missing the cache at the same moment and all hitting the database for the same value*, a change-data-capture stream to invalidate or refill entries under a bounded-staleness contract, and **consistent hashing** ≈ *a way of assigning keys to servers so that adding or removing a server moves only a small fraction of keys* to give each host its own slice of the keyspace.[15]

Load balancing turned out to be harder than expected. The tier mixes hosts from about 26 to about 126 cores, and a control-plane balancer adjusts ServiceRouter weights based on CPU, with damping, clamping and a change throttle so shards don't get reshuffled constantly. Meta's lesson: a single fixed balancing policy can't handle both calm and stressed tiers, so it's moving to classify tier states and apply a matching policy to each.[15]

The gateway even took over transaction bookkeeping from the thick client. ZippyDB transactions are serializable per shard and use **optimistic concurrency control** ≈ *letting transactions run without locks and checking for conflicts at commit time, aborting one if they clash*.[17] Meta consolidated two parallel implementations behind a flag in nine phases, reaching 100% of transaction traffic without a reliability regression.[15]

How they migrated without touching client code

Traffic moves to ZGateway through client-side configuration flags, scoped by service and shard prefix, with a percentage ramp, a region filter and a global **kill switch** ≈ *a single configuration flag that instantly routes all traffic back to the old path if something goes wrong*. No client code changes are needed, and every step can be reversed.[15] It's the same pattern Grab used for its counter migration (Story 2 in issue #7): ramp by percentage, keep an instant way back.

WHAT IT MEANS FOR YOU

You probably don't have a million clients. But the reasoning in this post applies at much smaller scale, and it's worth stealing.

Count connections as a function of things you don't control. If your database's connection count is `instances × pool size` and another team owns the instance count (autoscaling web tier, serverless functions, a new batch job), you have Meta's problem in miniature. Postgres is especially sensitive, which is why PgBouncer exists. The fix is the same shape: a tier you operate whose fan-in you set.

A proxy moves the problem; it doesn't delete it. Meta says so directly: "Connection management does not disappear behind a proxy; it moves to the one place we can solve it."[15] The gateway itself now needs capacity planning, balancing and its own overload protection. Meta spent real effort on all three.

The hop is a trade, not a free lunch. About 6% compute overhead and one more network hop buy bounded fan-in, cross-client batching and centralised controls.[15] Meta's view is that "the trade pays off when the client population is large, diverse, and not yours to change." When you own every client and they're few, a thick client or a library may still be better, which is why ServiceRouter keeps 99% of Meta's RPCs proxy-free.[18]

Order matters. Meta notes that direct access was the right early design, and the shared tier only became feasible after ServiceRouter improvements, Thrift overload protection and a thin client existed.[15] Don't build a gateway before you have the observability and overload protection to run one.

Per-tenant fairness is the feature you'll wish you had. A connection cap and per-tenant shedding under overload are different features, so it's worth checking which one your pooler or proxy actually gives you. If your database is shared by many services, the 6-of-1,350 result is the strongest argument in the post.

💥 Watch out for

- **The connection numbers are modelled.** The 19x and 97–98% figures come from a model with mock inputs, not production measurements.[15,16]
- **No latency numbers.** The post doesn't publish latency before and after, so the cost of the extra hop is given only as ~6% compute overhead.[15]
- **Caching changes consistency.** The read-through cache works under a bounded-staleness contract; reads that need strong consistency must not depend on it.[15,17]
- **Not open source.** ZGateway is internal Meta infrastructure; the lesson is the design, not a download.

WHAT HAPPENS NEXT

Meta lists four directions: route all ZippyDB traffic through the gateway; expose the control-loop parameters as a structured surface that AI agents can monitor and act on behind guardrails; co-locate parts of the gateway next to ZServer hosts for latency-sensitive workloads, while keeping connection management and admission control in the shared regional tier; and split the gateway into cooperating processes for stronger fault isolation.[15,16] Co-location is the interesting one: it's an admission that, for some workloads, the hop does matter, and the end state may be a hybrid rather than a pure middle tier.

📖 Words from this story

ZippyDB: Meta's general-purpose distributed key-value store, built on RocksDB, in production since 2013.

RocksDB: an embedded LSM-tree key-value storage engine from Meta.

Shards: independent slices of the keyspace, stored and replicated separately.

Paxos: a consensus protocol that lets replicas agree on the order of writes despite failures.

ServiceRouter: Meta's service mesh, implemented mostly as a library linked into services.

Thick client: a client library containing routing, shard maps, replica selection and batching.

File descriptor: an OS handle for an open file or socket, with a per-process limit.

OOMs: out-of-memory kills of processes that exhaust memory.

PgBouncer: a connection pooler that lets many Postgres clients share few server connections.

Admission control: deciding at the door whether to accept a request, based on load and quota.

Hedging: sending a duplicate request to a second replica and taking the first answer.

Linger window: a short wait to collect more requests into one batch.

Coalescing: merging identical concurrent requests into one backend fetch.

Hot key: a key receiving a disproportionate share of traffic.

Tenant: one customer or use case sharing a system's capacity.

Goodput: the rate of useful, successful work.

Read-through cache: a cache that fetches from the database itself on a miss.

Thundering herds: many simultaneous cache misses all hitting the database for the same value.

Consistent hashing: assigning keys to servers so membership changes move few keys.

Optimistic concurrency control: running transactions without locks and checking for conflicts at commit.

Kill switch: one flag that instantly routes traffic back to the old path.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **PostgreSQL 19 dates:** the open-items wiki now lists RC1 for 15 October and a planned GA on 29 October 2026, so a follow-up to issue #1 is close.[14]
- **Kafka 4.4.0 still pending:** the official downloads page's newest release by date is still 4.2.2 (29 Sep), with 4.3.1 (25 Jun) the highest version.[20]
- **Confluent Freight clusters grow up:** since 2 Oct they support key-based compacted topics, the idempotent producer, transactions and `DELETE_RECORDS`, narrowing the gap with standard Kafka clusters.[21]
- **Native Rust Kafka client:** Confluent put a Rust producer, consumer and admin client into preview on 1 Oct, and shipped librdkafka 2.16.0-based clients (free-threaded Python among them) on 7 Oct.[21]
- **Iceberg's Kafka Connect sink on notice:** a Confluent connector hub notice warned it faces removal unless flagged issues are fixed; Iceberg PR 18019 fixes its LICENSE/NOTICE files and drops the Hive distribution.[22]
- **Snowflake governance UI:** a Snowsight interface for managing tags and data protection policies went GA on 9 Oct, followed by a 12 Oct note on named workload identity management for users.[23]
- **Databricks buys Row Zero:** the cloud-spreadsheet start-up (24 Sep, terms undisclosed) will pair a spreadsheet front-end with Genie, and Ali Ghodsi says "many more acquisitions" are coming.[24]
- **SQLDoom:** Lukas Vogel ported Doom's game logic and renderer to pure SQL on CedarDB, a compiling database, with deathmatch working.[25]

---

1. **Introducing WalShadow: Sub-second Postgres replication to ClickHouse from physical WAL**, Sai Srirampur, ClickHouse, 10 Sep 2026.
2. **WalShadow: sub-second Postgres to ClickHouse replication**, ClickHouse product page, viewed 12 Oct 2026.
3. **ClickHouse/walshadow**, GitHub repository and README, viewed 12 Oct 2026.
4. **ClickHouse's WalShadow replicates Postgres to ClickHouse in 200 ms from physical WAL**, Zeli summary of the Hacker News thread, 11 Sep 2026.
5. **The Shadow Knows**, Christophe Pettus (PGX Inc.), The Build, 22 Sep 2026.
6. **walshadow-101: review + hand-rolled WAL decoder demo**, glukicov, GitHub, viewed 12 Oct 2026.
7. **Write Ahead Log configuration (wal_level, full_page_writes, wal_log_hints)**, PostgreSQL 19 documentation.
8. **Logical Decoding Concepts**, PostgreSQL 18 documentation.
9. **Postgres 9.4 feature highlight: REPLICA IDENTITY and logical replication**, Michael Paquier, 24 Apr 2014.
10. **Postgres REPLICA IDENTITY**, Illuminated Computing, Oct 2024.
11. **ClickHouse Acquires PeerDB to Boost Real-time Analytics with Postgres CDC Integration**, ClickHouse, 30 Jul 2024.
12. **ReplacingMergeTree**, ClickHouse documentation.
13. **TOAST**, PostgreSQL documentation.
14. **PostgreSQL 19 Open Items**, PostgreSQL wiki, viewed 12 Oct 2026 ("Important Dates").
15. **ZGateway: Learnings from Putting a Proxy in Front of ZippyDB**, Rittik Banik and Yunhao Cao, Engineering at Meta, 3 Sep 2026 (updated 8 Sep).
16. **Meta's ZGateway Cuts ZippyDB Connections 19x While Handling 1B+ Operations per Second**, Leela Kumili, InfoQ, 28 Sep 2026.
17. **Building a general purpose key value store for Facebook with ZippyDB**, Engineering at Meta, 6 Aug 2021.
18. **ServiceRouter: Hyperscale and Minimal Cost Service Mesh at Meta**, Harshit Saokar et al., OSDI 2023.
19. **Redis by proxy: Percona targets the last major hurdle to Valkey adoption**, Paul Sawers, The New Stack, 8 Oct 2026.
20. **Apache Kafka downloads**, Apache Kafka, viewed 12 Oct 2026.

21. **Release Notes for Confluent Cloud**, Confluent documentation, entries 1–9 Oct 2026.

22. **Apache Data Lakehouse Weekly: September 29 to October 7, 2026**, Alex Merced, 7 Oct 2026.

23. **Snowflake server release notes and feature updates**, Snowflake documentation, viewed 12 Oct 2026.

24. **Databricks buys Row Zero and is scouting for more startups to acquire**, TechCrunch, 24 Sep 2026.

25. **Clever database, but can it run Doom?**, Richard Speed, The Register, 3 Oct 2026.